

Monte Carlo Tree Search for 7 Wonders

Ryan Quinlan
Computer Science
CU Boulder
ryan.quinlan@colorado.edu

Abstract—This document explores the performance of Monte Carlo Tree Search (MCTS) for the board game 7 Wonders. We draw conclusions about both the MCTS algorithm and the board game itself.

I. INTRODUCTION

7 Wonders is a popular strategy board game developed by Antoine Bauza in 2010. It has since become a staple in both competitive and casual circles within the gaming community. It remains an unsolved game even among tournament players, with much effort being put into statistical analysis of human games to reach conclusions about good strategies. It is currently thought of as a very tactical game, meaning that overarching strategies are considered less important than making the optimal choice in the current state. One of the reasons for this paper is to add to discussions surrounding 7 Wonders strategy through the use of computer engine analysis.

The other goal of this paper is to explore the performance of the popular Monte Carlo Tree Search algorithm on the game. We employ both a vanilla MCTS algorithm (with random rollouts) as well as an algorithm using heuristic informed rollouts. MCTS has long been a popular algorithm for playing perfect information games, even being employed by the cutting edge Go, Chess, and Shogi algorithm AlphaZero [4]. This approach is partially inspired by ZeusAI [2], an artificial intelligence system that has achieved top-level human performance for the 2-player perfect information version of 7 Wonders: 7 Wonders Duel. While we do not achieve similar performance on the multi-player imperfect information game, we are still able to draw conclusions both about the game itself and the construction of our algorithm.

Again, high performance was not achieved, but our player does routinely score solid average scores when playing against itself, players making random moves, and even human players. The code repository includes a playable demo where readers can pit themselves against the bot discussed in this paper. There is much future work to be done to bring performance more in line with that of ZeusAI, but we make good progress and provide a solid foundation to continue with.

II. THE GAME

In order to fully understand and explore the conclusions we draw from the MCTS algorithm, we must first understand the game itself.

A. Basic Rules

7 Wonders is a 3-7 player strategy board game played in a card-drafting format. Each player assumes the role of a leader of an ancient city (randomly assigned at the start of the game) and competes to develop their board by acquiring resources, constructing buildings, and bolstering their military might. The game consists of 3 rounds (eras) of 6 turns each, with each era having its own pool of increasingly powerful cards that players draw a 7-card hand from.

Each turn, players select a card to play (these are revealed simultaneously) which either provide victory points or resources which can be used to build more powerful cards later in the game. Instead, players may choose to discard a card for 3 gold or build the next tier of their wonder. Players must pay the cost of each card either by using the resources they have already crafted (which are replenished every turn) or paying adjacent players to use their resources.

The main ways to gain victory points are civilian cards (which give a set number of points), military cards (which give points at the end of eras for each neighbor you defeat), and science cards (which give exponentially growing points for building many science cards). The winner of the game is the player who has accumulated the most victory points by the end of era 3.



Fig. 1. Example of a cards.

B. Observations

There are a few observations to be made about the game that will be relevant in our analysis. First, while the game is an imperfect information game, it is not fully imperfect in a few key ways. The pool of cards from which the players draw at the beginning of eras is fixed for a given player count: as in every 4 player game of 7 wonders will have the same 28 cards in it, they will just be distributed differently. This is not the case in many imperfect information games such as poker, where there is a pool of 52 cards, but only $5 \cdot n + 5$ cards are used in a given game. Additionally, the game becomes a perfect information game after n turns, where n is the number of players. This is due to the rotating hands cards are played from. For example, on turn 2 of a 3 player game, I know what cards I and my neighbor to the left have. On the next turn, I know what cards are in each players hand. This makes bluffing a significantly smaller portion of common human strategies (one of the reasons other common algorithms such as Counter-Factual Regret Minimization were not employed).

Second, strategies greatly depend on the number of players in the game. One of the often discussed aspects of the game is the efficacy of blocking your neighbors' strategies vs fully focusing on your own. In a 3 player game, blocking becomes more relevant because of course there are fewer opponents. Since you need to beat 2 players instead of 6 for example, disrupting one players strategy is 3x as effective.

III. THE ALGORITHM

Monte Carlo Tree Search is a widely-used algorithm for decision making in games, often used when decision spaces are large and a Nash equilibrium does not exist. Our implementation adapts the algorithm to handle multiplayer imperfect information games and adds a heuristic to the rollout step (discussed later). We choose MCTS because of its ability to handle the large decision spaces and its ability to explore many different paths. We use another groups implementation of the game [1], which essentially boils a state down to a group of players and their cards in hand and cards played.

Other attempts at MCTS for 7 Wonders [3] treat each individual action by each player as its own node but to moves being made simultaneously we choose to treat each new node as a complete game state with all actions taken. This simplifies the structure of the tree but we must keep in mind that transitions between nodes are actually decision sets containing actions for each player and not individual actions. Once a game is initialized, we construct the tree (just the root node for now) and proceed with the following phases of the algorithm.

First is **selection**, where we take the root node and choose the most "interesting" child node we want to explore further. We have chosen to use the Upper Confidence Bound policy:

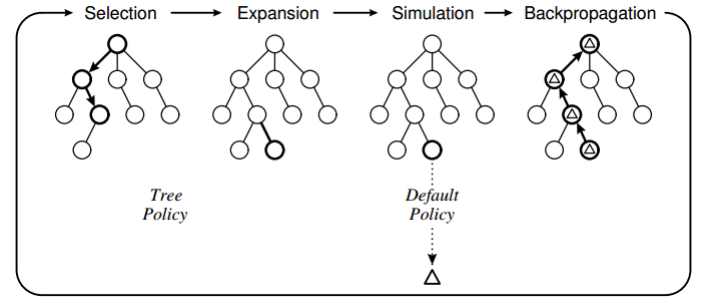


Fig. 2. Visualization of MCTS algorithm.

$UCT = \frac{Q}{N} + C \times \sqrt{\frac{\ln(N_p)}{N}}$ where Q is the total score of a node, N is the number of visits to the node, N_p is the parent's visited count and C is a constant. This is intended to inject a balance between simulating seemingly good moves more while still exploring seemingly bad ones. For simplicity, we chose $C = 1$ for our constant, but future work would involve turning this knob a bit.

Second is **expansion**, which involved generating child nodes for the selected node. Here, we apply all possible combinations of legal moves for each player in the game. For example, on turn 3 there will be $5^3 = 125$ children since there are 5 cards in each players hand. We create a new game state where a given combination is played and add that state as a child of the current selected node.

Next is **simulation**, which attempts to estimate the value of the node by performing a random rollout. In the vanilla version of the algorithm, we start from the selected node and apply random moves until a terminal state is reached. While performance was decent here, adding a heuristic to the rollout improved our algorithm significantly. Instead of players performing random moves, they instead perform actions according to a heuristic created based on advice of top human players. The heuristic rewards moves that increase a players score, resource, or military might as well as moves that block neighbors. In order to add some variety to our rollouts, we occasionally still choose a random move even in our heuristic aided rollouts. This is to prevent overfitting to the underperformant heuristic. We refer to this percentage chance of taking a random as the "exploration constant", not to be confused with the exploration constant from the UCT formula.

Finally is **backpropagation**, which takes the score achieved in the simulation and applies it to the selected node and its ancestors, all the way up to the root. Each of these nodes has their score increased by the points achieved in the simulation (Q) and has their visited count (N) incremented. We chose to use score instead of a binary win/loss metric but future works could explore using some combination of the two.

This process is repeated for a certain number of iterations,

then the child of the root node with the highest average value is selected as the move for the MCTS agent. For each move we create a new tree with the current state as the root and we maintain a separate tree for each player.

Vanilla MCTS only works for perfect information games. We solve this issue by using a modified version of Perfect Information Monte Carlo (PIMC), which has shown strong performance on other imperfect information card games such as hearts [5]. PIMC’s solution to MCTS’s issues with imperfect information is to determinize these states by sampling from the set of possible true states. This is generally great for games that draw from a deck, where not all cards are being used at any given time, making bluffing generally important. We modify this algorithm for simplicity by only sampling a single time. We do this to increase speed with the idea that 7 Wonders has less hidden information than usual since all cards in the pool are always in use. Future work could explore true sampling.



Fig. 4. Example of wonders.

both about the algorithm itself as well as the board game.

The first experiment ran is focused on the effects of certain hyperparameters on the performance of the algorithm. For this experiment, we pit the MCTS bot against players taking random moves, which act as a baseline from which we can improve upon. For example, in each of the the 3 player games in the results, there is 1 MCTS player with the indicated hyperparamter configuration and 2 players each taking random moves. The hyperparameters we focus on are player count (3, 5 and 7 player), search iterations (how many loops of the MCTS search algorithm we do per move), and exploration constant (remember, this refers to the chance of taking a random move instead of the heuristic defined best move during our rollouts). We run 100 games at each configuration and track win rate and average score. The point of this experiment is to determine the optimal configuration for our MCTS player and to see how that optimal configuration changes based on the player count. We can reach conclusions both about the MCTS algorithm and the game itself from these results.

The second experiment focuses on the strategies employed by our AI. In it, we take the strongest and weakest configurations for each player count and run 100 games against players taking random moves. In addition to tracking win rate and average score, we also track the strategies utilized. We are trying to determine if certain strategies such as science or military focus produce higher win rates or scores at certain player counts. We are also interested in whether or not these strategies depend on neighbor’s moves. For example, does a science focus still work if your neighbor is employing the same strategy?

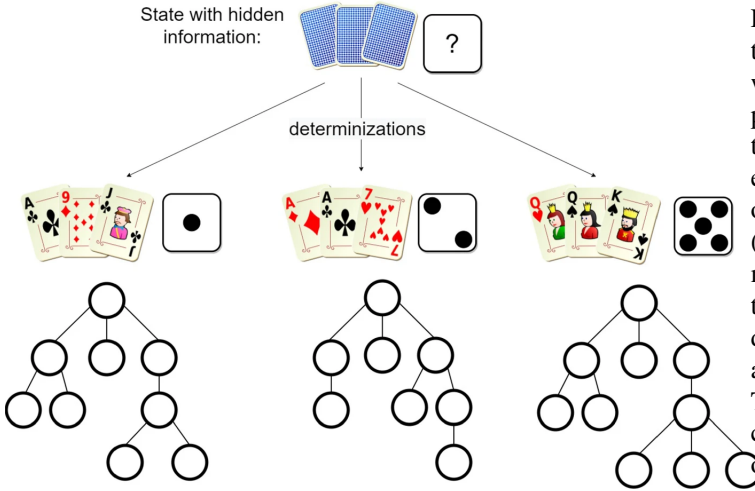


Fig. 3. Visualization of PIMC determinization.

One other shortcoming of our algorithm is our handling of wonders and discards. As discussed in the rules section, a player has the option to discard their card instead of playing it to either gain 3 gold or build a stage of their wonder (which is randomly assigned at the start of the game). Generally these are considered bad plays (especially discarding for gold) so our current algorithm only does these if there is no playable card. This is probably suboptimal but necessary as we currently represent valid moves simply as cards. Future work should expand this representation to include wonder building and discarding so they can be considered in the MCTS search instead of only played as default moves when no other move is available.

IV. METHODOLOGY

We run a variety of experiments to test the performance of our algorithm. Again, we are trying to reach conclusions

V. RESULTS

Raw values in appendix.

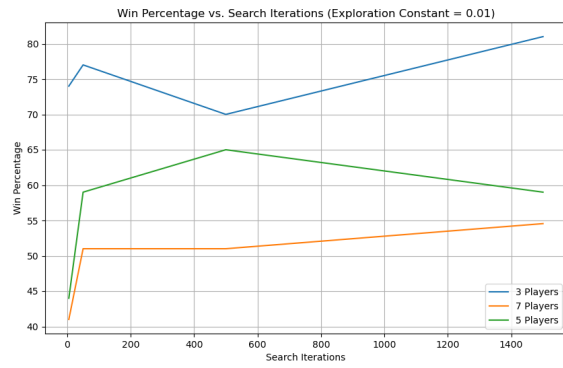


Fig. 5. Win rate vs search iterations for exploration constant of 0.1.

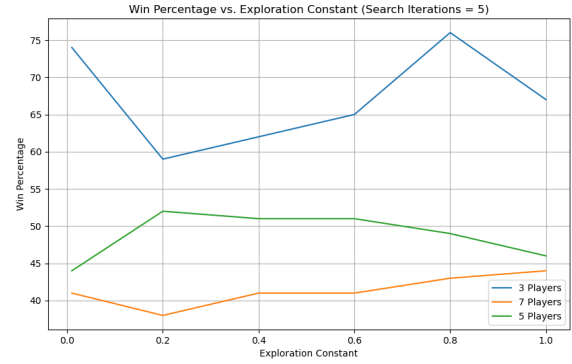


Fig. 8. Win rate vs exploration constant for 5 search iterations.

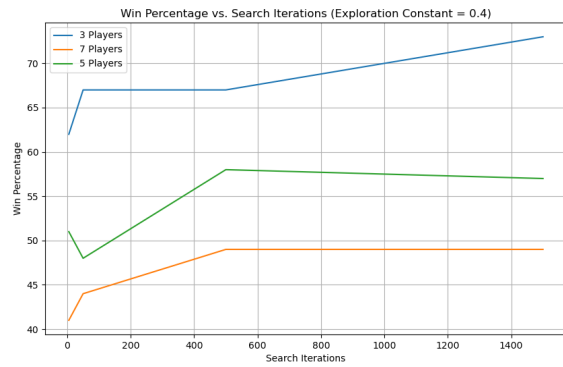


Fig. 6. Win rate vs search iterations for exploration constant of 0.4.

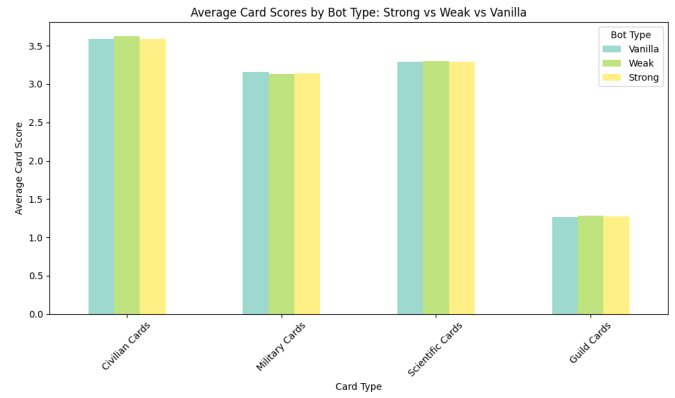


Fig. 9. Strategies for different bot types.

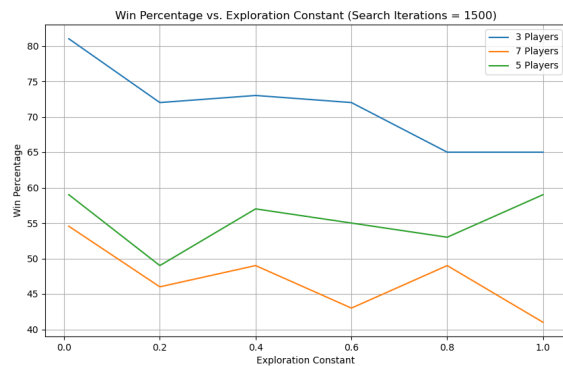


Fig. 7. Win rate vs exploration constant for 1500 search iterations.

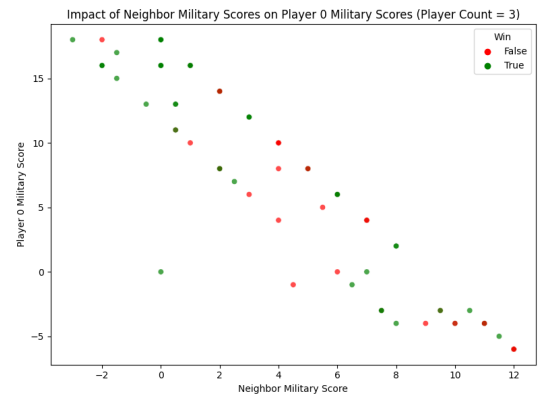


Fig. 10. Wins and losses based on military score and neighbors' military score at 3 players.

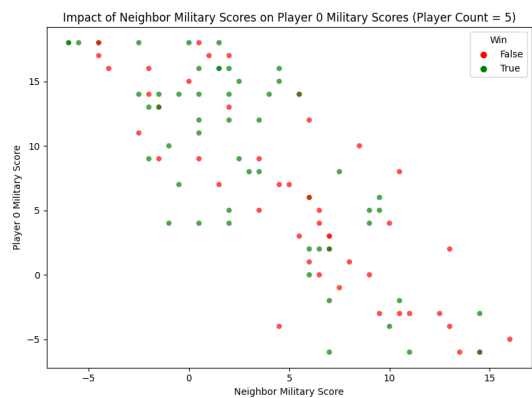


Fig. 11. Wins and losses based on military score and neighbors' military score at 5 players.

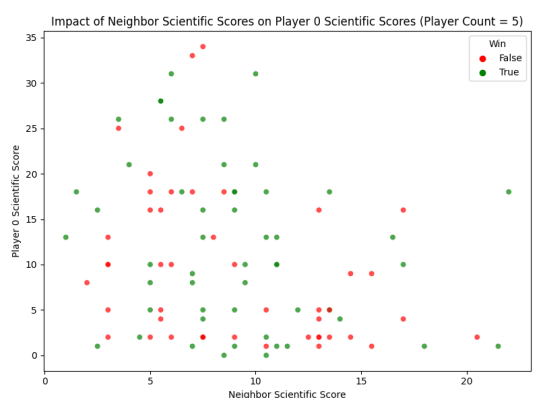


Fig. 14. Wins and losses based on science score and neighbors' science score at 5 players.

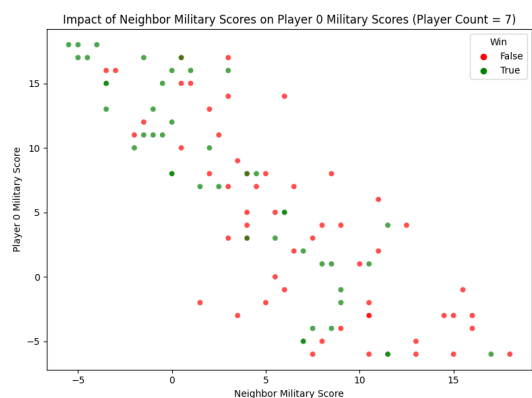


Fig. 12. Wins and losses based on military score and neighbors' military score at 7 players.

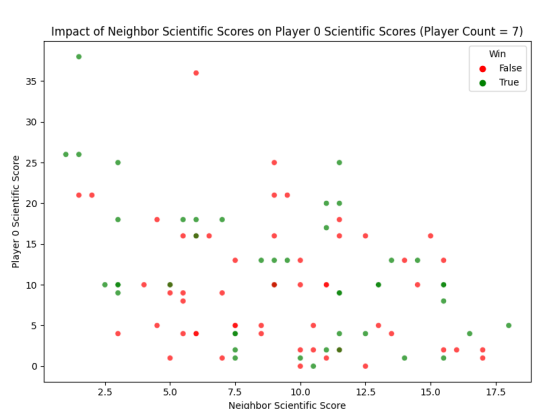


Fig. 15. Wins and losses based on science score and neighbors' science score at 7 players.

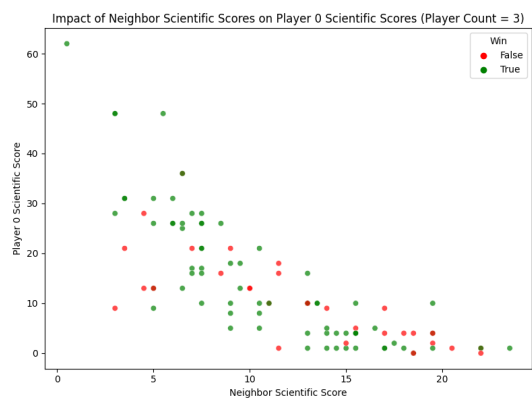


Fig. 13. Wins and losses based on science score and neighbors' science score at 3 players.

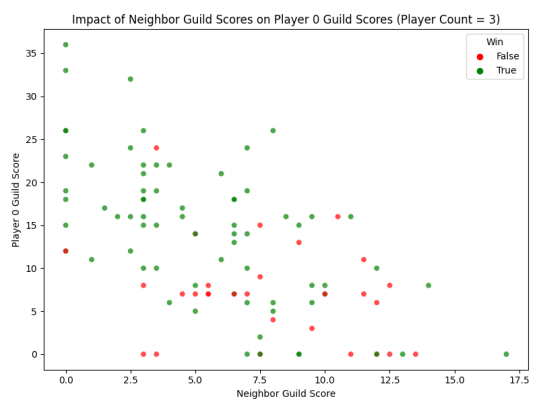


Fig. 16. Wins and losses based on guild score and neighbors' guild score.

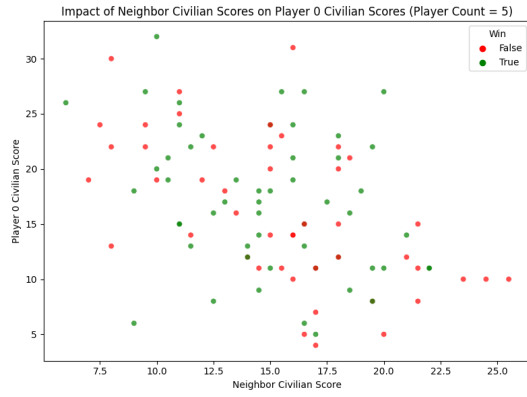


Fig. 17. Wins and losses based on civilian score and neighbors' civilian score.

A. Experiment 1

B. Experiment 2

VI. ANALYSIS

A. Experiment 1

The relationship between search iteration and win rate seems to be roughly logarithmic in nature regardless of player count as we can see from Figure 5 and Figure 6. This implies that a deeper search would produce more accurate moves but potentially have diminishing returns for large values after a certain points. While all the plots are roughly logarithmic, we can see that 3-player games require deeper searches. This may be due to the fact that they are closer to perfect information games and are less complex, so we can more fully explore the decision space (especially a few turns into each era). To generalize this statement, it seems that deeper searches may be more beneficial the closer your game is to a perfect information game while using MCTS.

In contrast, the relationship between exploration constant and win rate seems to be slightly negative but fairly flat regardless of search iterations and player count. This may indicate that our heuristic is fairly weak and could use more fine tuning (or it could be replaced by a neural network). It seems that using the heuristic for every move in the rollout is marginally beneficial but not crucial to the performance of the algorithm. To generalize, it seems that adding more realistic moves to your rollouts in MCTS provides a benefit to performance, but a fair amount of effort needs to go into that heuristic before it becomes significantly better.

B. Experiment 2

There are quite a few insights we can glean from the data surrounding the strategies our bots are going for. While a lot of them are in line with the common wisdom already thought to be true, it is good to have data confirming these preconceptions. For each player count, we run 100 games with the strongest bot for that player count pitted against bots taking random moves. We then analyze the typical avenues our MCTS bot went for and whether those strategies led to

wins. We also analyze the strategies of our neighbors to see if they have any affect on a winning game plan.

First, we can see that no one strategy is an end all be all. Fig 9 shows that the strong and weak bots for each player count generally go for the same strategies. This means that players must stay flexible and not go for one strategy (such as science) every game. This is in line with what top human players believe to be best practice.

Second, we can glean information about the validity of contesting military strength with your neighbors. The general consensus is that building military cards while your neighbor is strong is a bad idea. This is borne out in the data for 3 players, which shows higher win rates for both high and low military scores. However, when the player and its neighbors have mediocre scores (which represents a contested military board), the player's win rate drops significantly. This does not hold true at higher player counts, where regardless of neighbor strategy it is beneficial to have some military strength.

Third, we can do the same analysis on science. Here, it seems that neighbor strategy is entirely uncorrelated with success. High science scores generally lead to wins, but in the low to mid range there is not a big difference (which corroborates our first observation). This seems to hold true at all player counts.

Finally, we can see in figures 16 and 17 that guild and civilian win rates are not correlated with neighbors strategy. In fact, it seems that civilian score is almost entirely uncorrelated with win rate. However, high guild scores almost always lead to wins. This can be incorporated into human strategies with a focus on capitalizing on guild cards when they drawn instead of passing them over for others like civilian or military.

C. Conclusion

This paper provides a solid base for 7 Wonders AI. While much work remains to be done in optimizing the MCTS algorithm for this game specifically, it does show promise. Future improvements could see superhuman performance and provide more insights into optimal strategies at different player counts. The unique partial imperfect information nature of the game provides a novel space for game-playing algorithms to explore.

REFERENCES

- [1] Rafael Bettker, Pedro Minini, Gabriel Pereira, Júlia Acosta, Joaquim Assunção, and Julio Pinto Coelho Ribeiro Jardim. An implementation of the 7 wonders board game for ai-based players. 11 2020.
- [2] Giovanni Paolini, Lorenzo Moreschini, Francesco Veneziano, and Alessandro Iraci. Learning to play 7 wonders duel without human supervision. 2024.
- [3] Denis Robilliard, Cyril Fonlupt, and Fabien Teytaud. Monte-carlo tree search for the game of "7 wonders". In *Proceedings of the Third International Conference on Computer Games, CGW14*, page 64–77, Berlin, Heidelberg, 2014. Springer-Verlag.

- [4] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy P. Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. *CoRR*, abs/1712.01815, 2017.
- [5] Daniel Whitehouse, Edward Powley, and Peter Cowling. Determinization and information set monte carlo tree search for the card game dou di zhu. pages 87 – 94, 10 2011.

VII. APPENDIX

Players	Search Iterations	Exploration Constant	Win Percentage	Average Score
3	50	1	71	57.48
3	50	0.01	77	56.93
3	1500	0.01	81	56.81
3	500	1	72	56.8
3	1500	0.4	73	56.51
3	1500	0.6	72	56.05
3	500	0.01	70	56.01
3	50	0.8	75	55.85
3	500	0.6	74	55.52
3	500	0.4	67	55.51
3	5	0.01	74	55.42
3	1500	0.2	72	55.39
3	50	0.6	66	54.93
3	5	0.8	76	54.8
3	50	0.2	68	54.71
3	500	0.8	66	54.36
7	1500	0.01	54.55	54.32
3	1500	1	65	54.21
3	5	1	67	53.95
3	50	0.4	67	53.94
3	1500	0.8	65	53.67
7	500	0.01	51	53.63
5	500	0.01	65	53.53
3	5	0.2	59	53.47
3	5	0.4	62	53.45
3	5	0.6	65	53.37
3	500	0.2	65	53.07
5	1500	0.6	55	52.89
5	50	0.01	59	52.72
7	1500	0.4	49	52.63
7	500	0.6	50	52.53
5	1500	1	59	52.45
5	500	0.2	60	52.4
7	500	0.4	49	52.36
7	500	1	48	52.13
7	50	0.01	51	52.07
7	500	0.8	43	51.95
5	500	0.6	55	51.81
7	500	0.2	45	51.81
5	500	0.4	58	51.8
5	1500	0.4	57	51.57
7	1500	0.2	46	51.55
7	50	0.2	46	51.54
5	50	0.6	52	51.49
7	1500	0.8	49	51.45
7	50	0.4	44	51.41
5	500	1	57	51.25
7	5	1	44	51.14
7	50	0.8	45	51.09
7	50	1	46	51.09
7	5	0.6	41	51.01
5	1500	0.01	59	50.92
7	1500	0.6	43	50.87
5	50	0.8	59	50.82
7	5	0.01	41	50.79
5	50	1	53	50.78
5	5	0.8	49	50.54
7	1500	1	41	50.4
5	50	0.4	48	50.35
7	5	0.2	38	50.3
5	1500	0.8	53	50.28
5	5	0.2	52	50.25
7	5	0.8	43	50.09
5	500	0.8	47	49.93
5	5	0.01	44	49.9
5	1500	0.2	49	49.84
5	5	0.6	51	49.82
7	5	0.4	41	49.68
5	5	1	46	49.1
7	50	0.6	35	49.09
5	5	0.4	51	48.99
5	50	0.2	52	48.93